

Universidade de São Paulo (USP)
Escola de Artes, Ciências e Humanidades

EP - Sistema Distribuído PeerSpot

Disciplina: Desenvolvimento de Sistemas de Informação
Distribuídos

Professor: Renan Cerqueira Afonso Alves

Alunos:

Bruno Hideo Ionedá — 15573619

Guilherme Samuel Lemos Segura — 15575611

Higor Rangel Viani Lopes — 15552946

João de Melo Fantini — 15462550

São Paulo, março de 2026

Conteúdo

1	Dos Modelos de Arquitetura Estudados, algum se encaixa?	3
1.1	Por que não cliente-servidor puro?	3
1.2	Por que não P2P estruturado com DHT?	3
1.3	Por que P2P Híbrido com tracker?	3
2	Arquitetura do Sistema	3
2.1	Componentes e Responsabilidades	3
2.2	Fluxos Principais	4
3	Como o Sistema Será Testado	4
3.1	Testes Funcionais Locais	4
3.2	Testes de Distribuição	5
3.3	Testes de Tolerância a Falhas	5
4	Arquitetura de Software Interna	5
4.1	Servidor de Metadados / tracker	5
4.2	Peer / Cliente	5
5	Faz Sentido Usar Algum Tipo de Middleware?	6
6	Comunicação	6
6.1	Síncrona ou Assíncrona?	6
6.2	Protocolos de Transporte	7
6.3	Tipo de Conexões	7
7	Protocolos e Mensagens	7
7.1	Formato	7
7.2	Tipos de Mensagem	7
8	Sincronização entre os Trackers — Multicast	9
8.1	Implementação do Multicast	9
8.2	Adição dinâmica de trackers	9
8.3	Vantagens em relação ao anel circular	10
8.4	Gerenciamento de presença dos peers	10
9	Diagrama de Arquitetura	10
10	Diagramas de Sequência	11
10.1	Upload de Música	11
10.2	Busca e Download de Música	12
11	Nomeação	12
11.1	Quais recursos precisam ser nomeados	12
11.2	Esquema de Nomeação	13
11.3	Mecanismo de Resolução de Nomes	13

12 Processos	14
12.1 Faz sentido usar threads?	14
12.2 Servidores stateful ou stateless?	14
12.3 Faz sentido usar técnicas de virtualização?	14
13 Sincronização	15
13.1 Relógio Real com NTP	15
13.2 Resolução de Conflitos — Last Write Wins	16
13.3 Remoção de Peers — Registro Tombstone	16
13.4 Relatório Periódico de Seed pelos Peers	17
14 Exclusão Mútua Distribuída	17
14.1 É necessária?	17
14.2 Exclusão mútua local vs. distribuída	17
14.3 Conclusão	18
15 Tolerância a Falhas de Tracker	18
15.1 Não há tracker líder, não há necessidade de eleição	18
15.2 Fallback no peer	18
15.3 Reintegração do tracker e balanceamento de carga	18

1 Dos Modelos de Arquitetura Estudados, algum se encaixa?

O modelo que melhor se encaixa para o PeerSpot é o **P2P Híbrido Não Estruturado com tracker**, inspirado no modelo descrito por Tanenbaum (Cap. 2.4) e exemplificado pelo BitTorrent (Cap. 2.4.4). Nessa arquitetura, a rede é composta por três tipos de componente: **peers comuns**, **tracker** e **tracker distribuídos**, sendo que, no modelo adotado, os tracker acumulam também a função de tracker.

1.1 Por que não cliente-servidor puro?

A arquitetura cliente-servidor ancora todo o sistema em um único componente central, que precisa ser robusto o suficiente para atender a todos os usuários simultaneamente. Em um serviço de compartilhamento de arquivos de áudio, o servidor central se tornaria rapidamente um gargalo de largura de banda e um ponto único de falha, exatamente as “falsas premissas” que Tanenbaum enumera na seção 1.4 (assumir que a banda é infinita e que a rede é confiável).

1.2 Por que não P2P estruturado com DHT?

O P2P estruturado com DHT garante lookup em $O(\log N)$ e elimina componentes centralizados, mas introduz overhead de manutenção das tabelas de roteamento e redistribuição de chaves a cada entrada ou saída de nó. Para nosso trabalho focado em transferência de áudio, essa complexidade de implementação não se justifica frente aos benefícios.

1.3 Por que P2P Híbrido com tracker?

A arquitetura híbrida trás ótimos benefícios: os tracker centralizam o índice de forma distribuída (sem ponto único de falha), enquanto a transferência dos arquivos ocorre diretamente entre os peers, sem passar por nenhum intermediário. Isso traz:

- **Escalabilidade:** a carga de armazenamento e transferência é distribuída entre os peers. Adicionar usuários aumenta a capacidade da rede.
- **Tolerância a falhas:** como os tracker sincronizam seus índices entre si via multi-cast, a queda de um tracker não torna os arquivos inacessíveis.
- **Eficiência de busca:** peers comuns não precisam manter tabelas de roteamento complexas, delegam a busca ao tracker ao qual estão conectados.

2 Arquitetura do Sistema

2.1 Componentes e Responsabilidades

Peer Nó básico da rede. Armazena chunks de arquivos de áudio localmente. Mantém uma conexão duradoura com seu tracker. Ao fazer upload de uma música, notifica o tracker com **nome** + **hash**. Para baixar, consulta o tracker, recebe a lista de peers que possuem os chunks e realiza a transferência diretamente via TCP.

tracker. Agrega um cluster de peers comuns. Mantém duas estruturas de índice:

- `nome_da_musica` $\rightarrow$ `hash`
- `hash` $\rightarrow$ lista de peers que possuem o arquivo

Responde a buscas dos peers do seu cluster e roteia para outros tracker quando necessário. Sincroniza suas tabelas com os demais tracker via **multicast TCP**.

Organização dos tracker. Os tracker se conhecem mutuamente e formam um grupo de multicast. Não há hierarquia entre eles todos são equivalentes e mantêm cópias sincronizadas do índice global.

2.2 Fluxos Principais

Upload de música

1. Peer faz hash do arquivo localmente.
2. Peer envia `REGISTER_FILE(nome, hash)` ao seu tracker via REST.
3. tracker atualiza sua tabela local (`nome` $\rightarrow$ `hash` e `hash` $\rightarrow$ `[peer]`).
4. tracker envia `SYNC_TABLE(hash, peers)` via multicast TCP para os demais tracker.
5. Todos os tracker atualizam suas tabelas.

Busca e download

1. Peer envia `SEARCH_FILE(nome)` ao seu tracker via REST.
2. tracker busca `nome` $\rightarrow$ `hash` em sua tabela local.
3. Se não encontrado, roteia a busca para outros tracker.
4. tracker retorna `SEARCH_RESULT(hash, [ip:porta dos peers fonte])`.
5. Peer conecta diretamente ao(s) peer(s) fonte via TCP e solicita os chunks.
6. Após completar o download, peer envia `REGISTER_FILE(hash)` ao tracker, passando a ser também fonte do arquivo.

3 Como o Sistema Será Testado

3.1 Testes Funcionais Locais

Etapa inicial: simular múltiplos peers na mesma máquina utilizando processos ou *containers* Docker. O objetivo é validar as funcionalidades básicas, upload, registro no tracker, busca, download e reprodução, em um ambiente controlado, antes de introduzir variáveis de rede real.

3.2 Testes de Distribuição

Subir peers e tracker em máquinas distintas (ou VMs), verificando: localização de arquivos via índice distribuído, consistência das tabelas após sincronização por multicast, compartilhamento e visualização de playlists entre peers de clusters diferentes, e funcionamento do roteamento de buscas entre tracker.

3.3 Testes de Tolerância a Falhas

Derrubar tracker e peers deliberadamente para verificar:

- Se os arquivos continuam acessíveis por meio de outros peers que possuem os chunks.
- Se a sincronização entre tracker se mantém após a saída de um nó do grupo de multicast.
- Se peers órfãos reconectam corretamente ao tracker de backup após falha do tracker principal.

4 Arquitetura de Software Interna

Cada tipo de componente possui uma organização interna distinta, conforme descrito por Tanenbaum nos Capítulos 2.1 e 2.4.

4.1 Servidor de Metadados / tracker

Adota **arquitetura em camadas** (Cap. 2.1.1):

- **Camada de API REST:** recebe requisições dos peers (`REGISTER_FILE`, `SEARCH_FILE`) e dos demais tracker em caso de roteamento.
- **Camada de lógica de negócio:** processa autenticação, gerenciamento de playlists, lookup nas tabelas de índice e roteamento de buscas.
- **Camada de persistência:** banco de dados relacional para dados duráveis (usuários, playlists). As tabelas de índice (`nome→hash` e `hash→peers`) são mantidas em memória, pois mudam com frequência e devem ser reconstruídas a partir dos peers ativos após uma reinicialização.
- **Módulo de multicast:** responsável por enviar e receber mensagens `SYNC_TABLE` via TCP multicast para manter os índices sincronizados entre todos os tracker.

4.2 Peer / Cliente

Adota **arquitetura P2P** (Cap. 2.4): não há distinção de papel fixo, o mesmo nó baixa e serve chunks simultaneamente.

- **Interface de usuário (UI):** player de áudio integrado, busca de músicas.
- **Módulo P2P:** envia e recebe chunks de arquivos diretamente com outros peers via TCP.

- **Gerenciador de chunks:** controla quais partes de cada arquivo foram obtidas, coordena downloads paralelos de múltiplas fontes.
- **Armazenamento local:** chunks de áudio em disco, indexados pelo hash do arquivo.

5 Faz Sentido Usar Algum Tipo de Middleware?

Sim. O Tanenbaum (Cap. 2.2) define *middleware* como a camada que abstrai a heterogeneidade da rede, permitindo que componentes distribuídos se comuniquem sem conhecer os detalhes uns dos outros. No PeerSpot, o middleware se manifesta em dois locais:

Entre peer e tracker. Um framework de comunicação padronizado, REST sobre HTTP, abstrai os detalhes de rede e permite que peers em plataformas diferentes (Windows, Linux, macOS) se comuniquem com o tracker de forma uniforme. Bibliotecas como Flask ou FastAPI (Python) atuam como middleware nessa camada.

Entre peers (transferência de chunks). O protocolo de troca de chunks sobre TCP padroniza como dois peers negociam quais partes de um arquivo um tem e o outro precisa. Uma biblioteca de abstração de sockets (como `asyncio` em Python) atua como middleware aqui, isolando o módulo P2P dos detalhes de baixo nível da comunicação.

O uso de middleware é especialmente justificável porque o sistema tem **heterogeneidade**, peers podem rodar em sistemas operacionais distintos, e o middleware resolve exatamente esse problema de interoperabilidade.

6 Comunicação

6.1 Síncrona ou Assíncrona?

A escolha entre comunicação síncrona e assíncrona vai depender da operação:

- **Síncrona**, operações em que o peer precisa da resposta para continuar: busca de música (`SEARCH_FILE`) e registro de arquivo (`REGISTER_FILE`). O peer aguarda a resposta do tracker antes de prosseguir.
- **Assíncrona**, sincronização entre tracker via multicast TCP. O tracker que atualizou seu índice não aguarda confirmação dos demais, dispara o `SYNC_TABLE` e continua operando. Isso evita que a indisponibilidade temporária de um tracker bloquear os demais.

6.2 Protocolos de Transporte

Comunicação	Protocolo	Justificativa
Peer ↔ tracker	REST sobre TCP (HTTP)	Simplicidade, padronização, suporte em qualquer linguagem
tracker ↔ tracker (sincronização)	UDP Multicast	Eficiência: um envio alcança todos os tracker simultaneamente
Peer ↔ Peer (transferência de chunks)	TCP direto	Garante entrega ordenada e sem perdas para dados binários

6.3 Tipo de Conexões

- **Conexões duradouras:** entre peer e seu tracker. O peer mantém a conexão aberta para receber notificações (ex.: outro peer buscou um arquivo que você possui).
- **Conexões temporárias:** entre peers durante download/upload. São estabelecidas sob demanda, após a descoberta dos peers fonte, e encerradas ao término da transferência.

7 Protocolos e Mensagens

7.1 Formato

Todas as mensagens de controle (registro, busca, sincronização) utilizam **JSON** sobre HTTP/UDP, por ser legível, amplamente suportado e adequado para depuração em ambiente acadêmico. A transferência de chunks de áudio ocorre em **binário puro** sobre TCP, sem encapsulamento JSON, para não impor overhead desnecessário em dados já comprimidos.

7.2 Tipos de Mensagem

```
1 # Registro de musica (peer -> tracker)
2 REGISTER_FILE = {
3     "type": "REGISTER_FILE",
4     "nome_peer": "string",
5     "nome": "string",
6     "hash": "sha256_hex",
7     "tamanho": "int_bytes",
8     "n_chunks": "int"
9 }
10
11 # Busca de musica (peer -> tracker)
12 SEARCH_FILE = {
13     "type": "SEARCH_FILE",
14     "query": "string",
15     "ttl": "int"           # decrementado a cada roteamento
16 }
17
```



```

18 # Resposta com peers disponiveis (tracker -> peer)
19 SEARCH_RESULT = {
20     "type": "SEARCH_RESULT",
21     "resultados": [
22         {
23             "hash": "sha256_hex",
24             "nome": "string",
25             "peers": [
26                 { "nome_peer": "string", "ip": "string", "porta": "int"
27             ]
28         }
29     ]
30 }
31
32 # Sincronizacao entre trackers (multicast UDP)
33 SYNC_TABLE = {
34     "type": "SYNC_TABLE",
35     "origem": "tracker_id",
36     "timestamp": 1743550800.123, # Unix timestamp (NTP-sincronizado)
37     "entries": [
38         { "hash": "sha256_hex", "peers": ["nome_peer"] }
39     ]
40 }
41
42 # Requisicao de chunk (peer -> peer, TCP)
43 CHUNK_REQUEST = {
44     "type": "CHUNK_REQUEST",
45     "hash": "sha256_hex",
46     "chunk_index": "int"
47 }
48
49 # Peer saindo da rede (peer -> tracker)
50 PEER_LEAVE = {
51     "type": "PEER_LEAVE",
52     "nome_peer": "string"
53 }
54
55 # Notificacao de mudanca de IP (peer -> tracker)
56 UPDATE_IP = {
57     "type": "UPDATE_IP",
58     "nome_peer": "string",
59     "novo_ip": "string",
60     "porta": "int"
61 }
62
63 # Remocao de arquivo especifico (peer -> tracker)
64 PEER_LEAVE_FILE = {
65     "type": "PEER_LEAVE_FILE",
66     "nome_peer": "string",
67     "hash": "sha256_hex"
68 }

```

Listing 1: Definição dos tipos de mensagem

O campo `tvl` na mensagem `SEARCH_FILE` é decrementado a cada roteamento entre tracker, evitando que uma busca percorra a rede indefinidamente. A resposta de chunk (`CHUNK_DATA`) é enviada diretamente como payload sobre TCP, precedida por um cabe-

çalho contendo hash e índice do chunk e tamanho do payload.

8 Sincronização entre os Trackers — Multicast

A sincronização entre tracker utiliza **UDP Multicast** na camada de aplicação. Todos os tracker ingressam no mesmo grupo multicast (endereço 224.0.0.1, porta 5007). Quando um tracker atualiza seu índice, dispara uma mensagem **SYNC_TABLE** para o grupo, todos os demais recebem simultaneamente sem que o remetente precise conhecer os endereços individuais.

8.1 Implementação do Multicast

O multicast é implementado diretamente sobre UDP usando sockets POSIX padrão, disponíveis em qualquer sistema operacional moderno. Cada tracker executa dois papéis simultâneos: um *sender*, que envia **SYNC_TABLE** ao grupo quando atualiza seu índice, e um *listener*, que escuta o grupo em uma thread dedicada e aplica as atualizações recebidas.

```
1 import socket, struct
2
3 MULTICAST_GROUP = '224.0.0.1'
4 PORT = 5007
5
6 # Enviar atualizacao para o grupo
7 def enviar_sync(mensagem: dict):
8     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9     sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)
10    sock.sendto(json.dumps(mensagem).encode(), (MULTICAST_GROUP, PORT))
11
12 # Escutar atualizacoes do grupo (roda em thread dedicada)
13 def escutar_sync():
14     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
15     sock.bind(('', PORT))
16     mreq = struct.pack('4sL', socket.inet_aton(MULTICAST_GROUP),
17                        socket.INADDR_ANY)
18     sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
19     while True:
20         data, _ = sock.recvfrom(65535)
21         aplicar_sync(json.loads(data.decode()))
```

Listing 2: Esqueleto do módulo multicast

8.2 Adição dinâmica de trackers

Sim, é possível adicionar novos trackers dinamicamente. O protocolo de ingresso é:

1. O novo tracker entra no grupo multicast chamando **IP_ADD_MEMBERSHIP** — a partir desse momento ele já recebe todas as mensagens **SYNC_TABLE** futuras.
2. O novo tracker envia **TRACKER_REJOIN** ao grupo, sinalizando sua presença.
3. Os trackers ativos respondem com **FULL_SYNC**, enviando o estado completo de suas tabelas diretamente ao novo tracker via UDP unicast.
4. O novo tracker aplica o **FULL_SYNC** e passa a operar normalmente.

Não é necessário reconfigurar os trackers existentes — eles não precisam conhecer o endereço do novo tracker, pois a comunicação usa o endereço de grupo multicast, não endereços individuais.

8.3 Vantagens em relação ao anel circular

A alternativa de organizar os tracker em lista ligada circular para propagação sequencial exigiria: descoberta de vizinhos, detecção de falha do vizinho, reconstrução do anel após falha, e controle de loop com TTL ou ID de mensagem. O multicast elimina toda essa complexidade, um tracker entra no grupo e automaticamente recebe todas as atualizações.

8.4 Gerenciamento de presença dos peers

O PeerSpot assume que o gerenciamento do ciclo de vida dos arquivos é responsabilidade do próprio sistema — não há heartbeat de peer. A remoção de um peer do índice ocorre apenas por ação explícita: quando o peer encerra sua sessão normalmente, envia PEER_LEAVE ao tracker; quando remove uma música específica, envia PEER_LEAVE_FILE. Em ambos os casos o tracker propaga a remoção via SYNC_TABLE.

9 Diagrama de Arquitetura

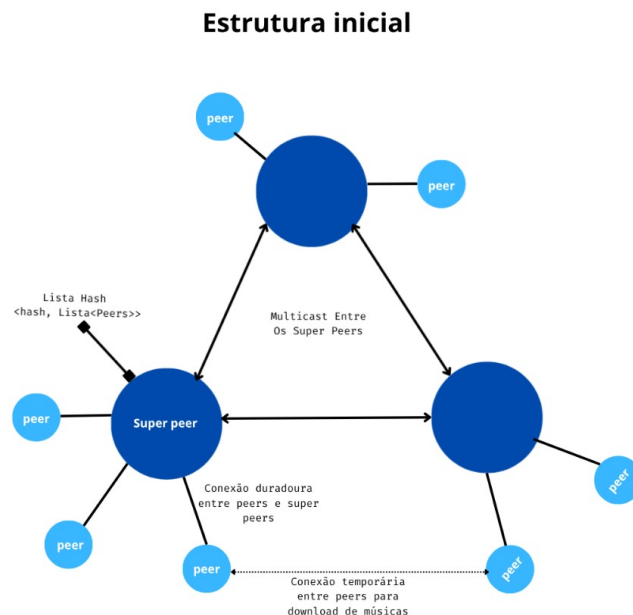


Figura 1: Diagrama de Arquitetura

10 Diagramas de Sequência

10.1 Upload de Música

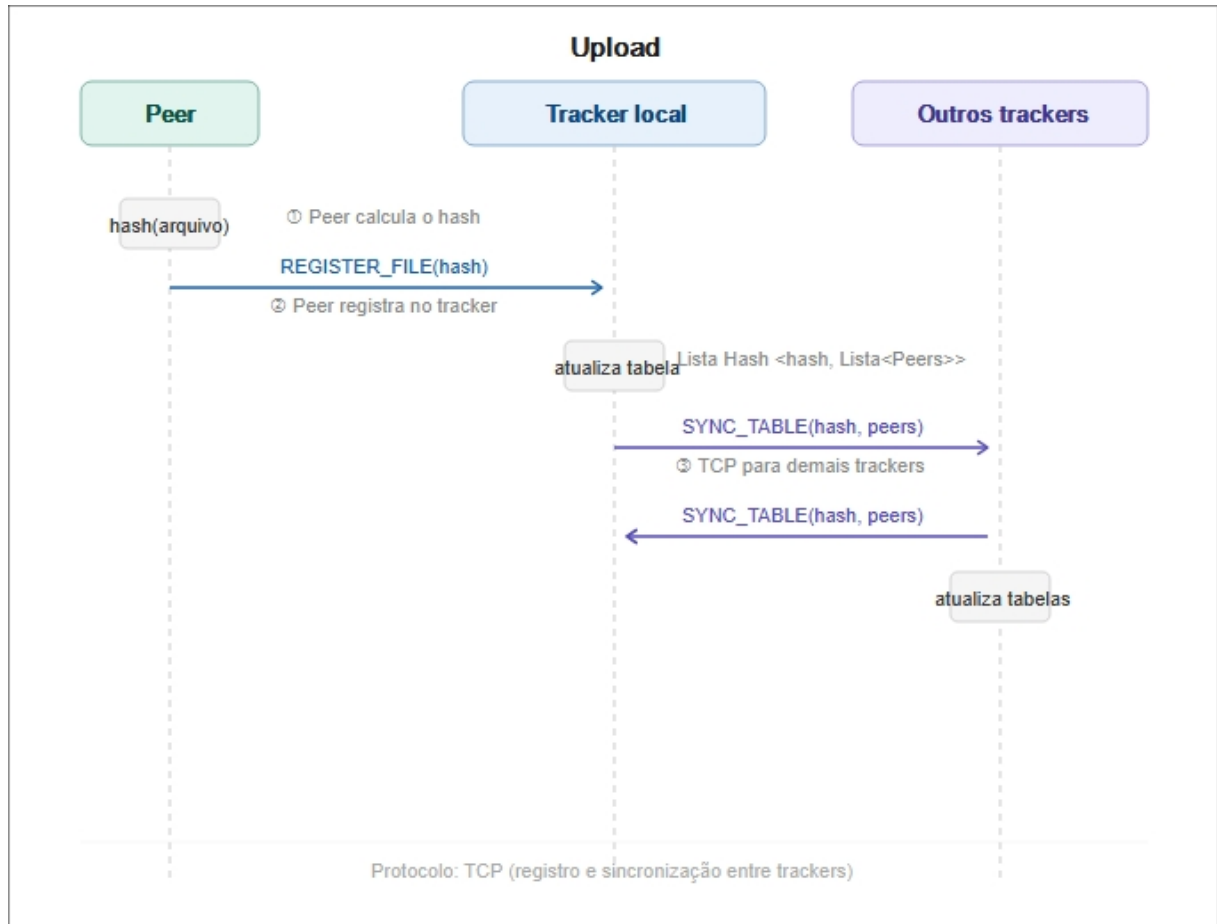


Figura 2: Diagrama de sequência do upload

10.2 Busca e Download de Música

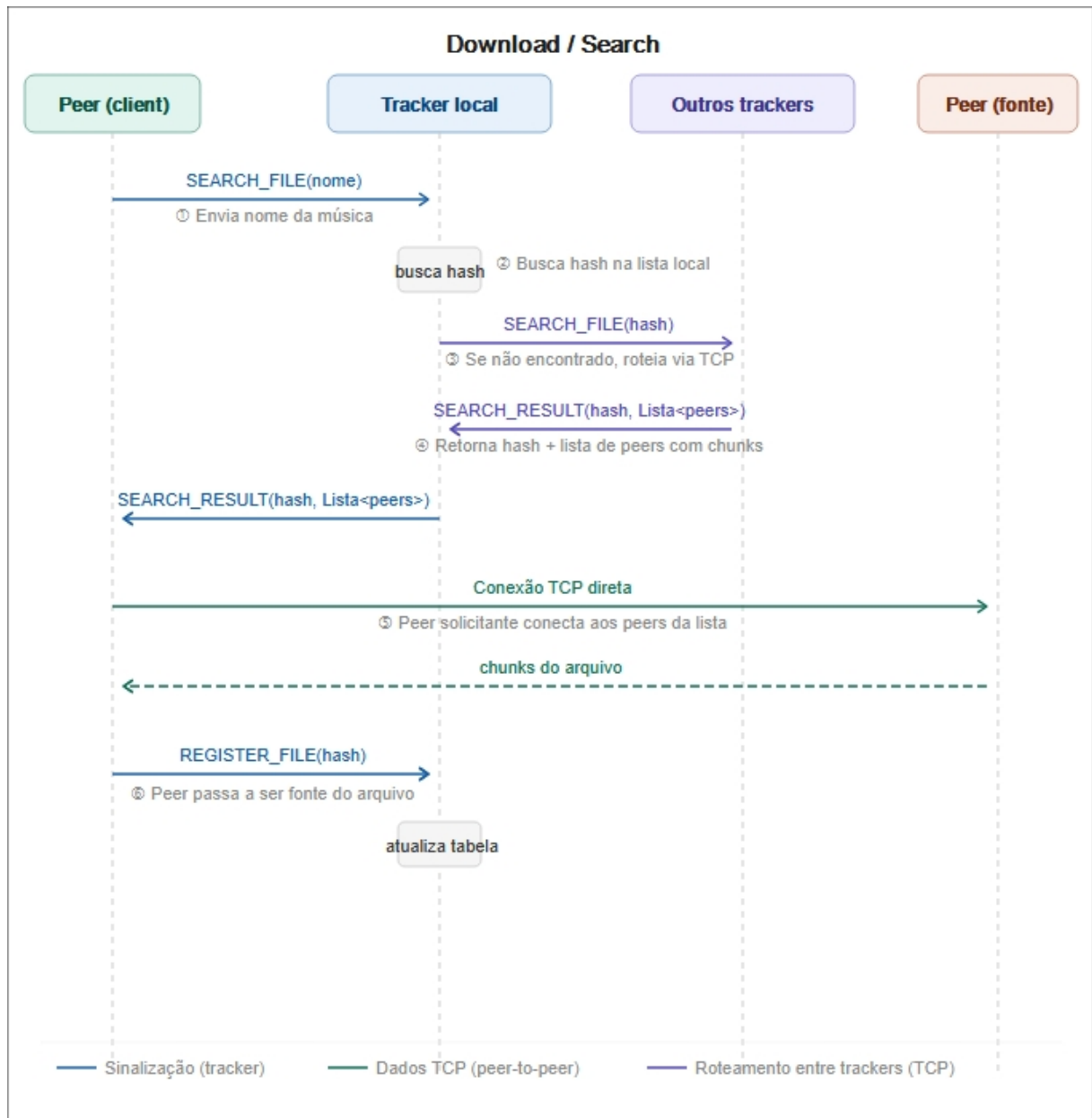


Figura 3: Diagrama de sequência do download

11 Nomeação

Tanenbaum (Cap. 5) define nomeação como o mecanismo pelo qual entidades de um sistema distribuído são identificadas e localizadas. No PeerSpot, três categorias de entidade precisam ser nomeadas: peers, arquivos de áudio e trackers.

11.1 Quais recursos precisam ser nomeados

- **Peers:** cada nó da rede precisa de um identificador único para que o tracker saiba quem possui quais arquivos e para que peers se conectem diretamente entre si durante a transferência.

- **Arquivos de áudio:** as músicas precisam ser identificadas de maneira única e independente de nome, dois arquivos com o mesmo nome mas conteúdo diferentes são entidades diferentes.
- **Trackers:** precisam se identificar entre si para controlar a origem das mensagens SYNC_TABLE e evitar processamento de atualizações duplicadas.

11.2 Esquema de Nomeação

O PeerSpot adota dois esquemas complementares, conforme a entidade:

Nomes estruturados — peers

Peers são identificados por um **nome de usuário** escolhido no cadastro, que deve ser único na rede. O tracker mantém uma tabela `nome_peer` → `ip:porta` que mapeia cada peer ao seu endereço de rede atual. Como peers domésticos podem ter IPs dinâmicos, toda vez que o IP de um peer muda ele notifica seu tracker com uma mensagem `UPDATE_IP`, que propaga a atualização via multicast para os demais trackers. Dessa forma, o nome do peer permanece estável como identidade, enquanto o IP é tratado como endereço de contato mutável.

```

1 UPDATE_IP = {
2     "type": "UPDATE_IP",
3     "nome_peer": "string",
4     "novo_ip": "string",
5     "porta": "int"
6 }
```

Listing 3: Notificação de mudança de IP

Nomes planos — arquivos

Arquivos de áudio são identificados pelo hash do seu conteúdo. Esse é um nome plano, uma sequência de 256 bits sem estrutura legível.

Nomes estruturados (legíveis ao usuário) — músicas

Para a interface do usuário, o sistema mantém no tracker um mapeamento de nome legível para hash: `nome_da_musica` → `lista de hashes`. Note que uma busca por nome pode retornar **múltiplos hashes**, pois é possível haver mais de uma música com o mesmo nome (artistas diferentes, versões distintas). Cabe ao usuário selecionar qual resultado deseja baixar. Internamente, toda operação usa o hash, o nome legível é apenas a camada de apresentação.

11.3 Mecanismo de Resolução de Nomes

A resolução ocorre em dois passos sequenciais, correspondendo às duas tabelas mantidas pelo tracker:

1. **Nome legível** → **lista de hashes:** o peer envia `SEARCH_FILE(nome)` ao seu tracker. O tracker consulta a tabela `nome` → `hashes` e retorna todos os identificadores correspondentes, permitindo que o usuário diferencie arquivos homônimos pelo hash ou por metadados adicionais.

2. **Hash → endereços de peers:** com o hash escolhido, o tracker consulta a segunda tabela (**hash → lista de peers**) e retorna os endereços IP e portas dos peers que possuem o arquivo. O peer solicitante usa esses endereços para estabelecer conexões TCP diretas.

Se o tracker local não possui o hash na tabela (arquivo registrado em outro cluster), ele roteia a consulta para os demais trackers via TCP, com decremento de TTL para evitar loops.

12 Processos

A seguir analisamos cada aspecto para o PeerSpot.

12.1 Faz sentido usar threads?

Sim, e em ambos os componentes.

No tracker. O tracker precisa atender múltiplos peers simultaneamente: receber registros de upload, responder buscas e processar mensagens de sincronização multicast. Um modelo single-threaded bloquearia todas as operações enquanto uma requisição estivesse sendo processada. A solução é um servidor multithread com pool de threads, onde cada requisição recebida é despachada para uma thread do pool.

Adicionalmente, o módulo de multicast roda em uma thread dedicada, escutando continuamente o grupo multicast em paralelo com o atendimento de requisições REST.

No peer. O peer precisa realizar simultaneamente: manter a conexão duradoura com o tracker, servir chunks para outros peers que estão baixando, baixar chunks de outras fontes, e manter a interface do usuário responsiva. Essas quatro atividades são naturalmente concorrentes e se beneficiam de threads independentes para cada responsabilidade.

12.2 Servidores stateful ou stateless?

Tracker Stateful. O tracker é stateful: mantém em memória as tabelas **nome → hashes**, **hash → peers** e **nome_peer → ip:porta**. O estado é essencial para a função do tracker, visto que nenhuma busca pode ser respondida sem ele.

A implicação é que, ao reiniciar, o tracker começa com as tabelas vazias e reconstrói o índice via **FULL_SYNC** com os demais trackers ativos.

Peer Parcialmente stateful. O peer é stateful no que diz respeito aos chunks armazenados localmente (persistidos em disco). A conexão duradoura com o tracker também é estado mantido em memória. Porém, as conexões temporárias de transferência entre peers são tratadas de forma stateless, dado que cada **CHUNK_REQUEST** é independente, e a conexão TCP pode ser encerrada e reestabelecida sem perda de progresso, pois o gerenciador de chunks controla quais índices já foram recebidos.

12.3 Faz sentido usar técnicas de virtualização?

Faria sentido em dois contextos distintos:

Durante o desenvolvimento e testes. O uso de containers Docker é a forma mais prática de simular múltiplos peers e trackers em uma única máquina durante o desenvolvimento. Cada container representa um nó da rede, com seu próprio endereço IP na rede virtual do Docker. Isso permite testar cenários de falha (derrubar um container), sincronização entre trackers e comportamento do multicast sem precisar de múltiplas máquinas físicas.

Em implantação real. Em um cenário de implantação, os trackers poderiam rodar em máquinas virtuais ou containers em provedores de nuvem, permitindo escalar o número de trackers conforme a demanda e recriar instâncias automaticamente em caso de falha.

Entretanto, dado o escopo do nosso projeto, tamanho trabalho não é viável por questões de tempo de implementação.

13 Sincronização

Tanenbaum (Cap. 6) distingue dois modelos de sincronização em sistemas distribuídos: relógios físicos (tempo real) e relógios lógicos (ordem de eventos). O PeerSpot adota **relógio real sincronizado via NTP**, conforme justificado a seguir.

13.1 Relógio Real com NTP

O PeerSpot utiliza **timestamps de relógio real** para ordenar os eventos entre trackers. Assume-se que todos os trackers têm NTP (*Network Time Protocol*) implementado e seus relógios sincronizados com um servidor de tempo confiável. Com essa premissa, os timestamps de tempo real são suficientes para estabelecer a ordem dos eventos: se dois SYNC_TABLE chegam sobre o mesmo arquivo, o de maior timestamp é o mais recente e vence (política LWW, descrita na seção seguinte).

A alternativa de relógios lógicos de Lamport também foi considerada, mas foi descartada: Lamport garante apenas ordenação causal (se A causou B , então $ts(A) < ts(B)$), sem ancoragem em tempo real. Como os trackers operam em rede local com NTP disponível, o desvio típico entre relógios é da ordem de milissegundos, suficiente para ordenar eventos de atualização de índice, cuja granularidade é de segundos. O relógio real é portanto mais simples e igualmente correto para o domínio do PeerSpot.

O campo `timestamp` da mensagem SYNC_TABLE carrega um timestamp Unix em ponto flutuante:

```
1 SYNC_TABLE = {
2   "type":      "SYNC_TABLE",
3   "origem":    "tracker_id",
4   "timestamp": 1743550800.123,  # Unix timestamp (NTP-sincronizado)
5   "entries": [
6     { "hash": "sha256_hex", "peers": ["nome_peer"] }
7   ]
8 }
```

Listing 4: SYNC_TABLE com timestamp real

13.2 Resolução de Conflitos — Last Write Wins

Mesmo com NTP sincronizando os relógios, é possível que dois trackers recebam atualizações conflitantes sobre o mesmo registro, por exemplo, dois peers em clusters distintos removem o mesmo arquivo quase simultaneamente, gerando duas mensagens `SYNC_TABLE` com timestamps diferentes chegando em ordens distintas aos demais trackers.

O PeerSpot adota a política **Last Write Wins (LWW)**: quando um tracker recebe uma atualização para uma entrada que já possui, compara o `timestamp` da mensagem recebida com o timestamp da versão local. Se o timestamp recebido for maior, a entrada local é sobrescrita. Se for menor ou igual, a mensagem é descartada como desatualizada. Essa política é simples de implementar e adequada para o domínio do sistema: o estado de um arquivo (quais peers o possuem) é naturalmente substituível pela versão mais recente.

A consequência aceita do LWW é que atualizações simultâneas com timestamps idênticos são resolvidas de forma arbitrária (por exemplo, pelo maior `tracker_id`). Dado que conflitos exatos são raros e o impacto é apenas a versão do índice de disponibilidade de um arquivo, não o arquivo em si, essa simplicidade é justificável para o escopo do projeto.

13.3 Remoção de Peers — Registro Tombstone

Um problema clássico em índices distribuídos é a remoção de entradas: quando um peer sai da rede ou remove um arquivo, o tracker local atualiza sua tabela e propaga via `SYNC_TABLE`. Porém, se outro tracker estava temporariamente desconectado e perde essa mensagem de remoção, continua servindo o peer removido como fonte válida, o que leva a tentativas de conexão falhas pelos clientes.

Para resolver isso, o PeerSpot adota o padrão **tombstone** (registro lápide): ao invés de simplesmente remover uma entrada, o tracker marca o peer como removido com um registro especial que persiste por um período configurado (ex.: 10 minutos):

```
1 # Entrada normal
2 { "hash": "sha256_hex", "nome_peer": "string", "ativo": True, "
   timestamp": 1743550800.1 }
3
4 # Tombstone: peer removido, entrada preservada temporariamente
5 { "hash": "sha256_hex", "nome_peer": "string", "ativo": False, "
   timestamp": 1743550860.3 }
```

Listing 5: Estrutura de tombstone no índice do tracker

Quando um tracker desconectado volta ao grupo multicast e recebe atualizações, ele aplica o LWW normalmente: se receber um tombstone com timestamp maior que sua entrada local, substitui a entrada ativa pelo tombstone, efetivando a remoção. Após o período de retenção, o tombstone é descartado definitivamente.

Por simplicidade, assume-se que o gerenciamento de arquivos é local ao peer — quando o usuário remove uma música, o peer notifica o tracker via `PEER_LEAVE_FILE`, que propaga a remoção aos demais trackers via `SYNC_TABLE`.

```
1 PEER_LEAVE_FILE = {
2     "type": "PEER_LEAVE_FILE",
3     "nome_peer": "string",
4     "hash": "sha256_hex"
5 }
```

13.4 Relatório Periódico de Seed pelos Peers

O PeerSpot adota um mecanismo de **relatório periódico de seed**: a cada **3 minutos**, cada peer envia ao seu tracker local a lista completa de hashes dos arquivos que ainda possui e para os quais está disponível para fornecer chunks.

```
1 SEED_REPORT = {  
2     "type": "SEED_REPORT",  
3     "nome_peer": "string",  
4     "ip": "string",  
5     "porta": "int",  
6     "hashes": ["sha256_hex", "sha256_hex"]  
7 }
```

Listing 6: Mensagem de relatório periódico de seed

Como o gerenciamento de arquivos é responsabilidade do sistema, o `SEED_REPORT` é o mecanismo pelo qual o tracker mantém seu índice de disponibilidade atualizado. Um peer pode remover músicas do disco local a qualquer momento; sem o relatório periódico, o tracker continuaria anunciando esse peer como fonte válida de arquivos que ele não possui mais, resultando em transferências falhas.

Ao receber um `SEED_REPORT`, o tracker compara a lista recebida com o que tem registrado para aquele peer. Hashes ausentes no relatório são marcados com tombstone e propagados via `SYNC_TABLE`. Hashes novos são registrados normalmente. O intervalo de 3 minutos equilibra a frequência de atualização do índice com o overhead de rede gerado pelas mensagens periódicas.

14 Exclusão Mútua Distribuída

14.1 É necessária?

Sim, em um cenário específico: a **atualização concorrente do índice no tracker**.

Cada tracker mantém em memória as tabelas `nome→hash` e `hash→peers`. Internamente, múltiplas threads podem tentar atualizar essas estruturas simultaneamente, uma thread atendendo um `REGISTER_FILE` de um peer local enquanto outra thread processa um `SYNC_TABLE` recebido via multicast. Sem exclusão mútua, essas escritas concorrentes podem corromper o estado do índice.

14.2 Exclusão mútua local vs. distribuída

É importante distinguir dois níveis:

Exclusão mútua local (dentro de um único tracker): resolvida com primitivas de sincronização da linguagem, `threading.Lock()` em Python, por exemplo. Isso protege as estruturas de dados em memória contra condições de corrida entre as threads internas do tracker. Não é um problema distribuído, é um problema de concorrência clássico dentro de um processo.

Exclusão mútua distribuída (entre múltiplos trackers): seria necessária se dois trackers pudessem modificar a mesma entrada do índice de forma conflitante. No PeerSpot, contudo, a semântica das operações evita esse problema: a operação de registrar um arquivo é *aditiva*, adiciona um peer à lista de fontes de um hash. Dois trackers adicionando peers diferentes ao mesmo hash de forma concorrente não gera conflito, pois a

operação de união de conjuntos é comutativa. Não há operações de exclusão concorrente que exijam coordenação entre trackers.

14.3 Conclusão

O PeerSpot **não necessita de exclusão mútua distribuída** entre trackers, pois as operações de atualização do índice são aditivas e comutativas. A exclusão mútua necessária é apenas local, dentro de cada tracker, resolvida com locks de threading padrão. Tanenbaum (Cap. 6.4) aponta que algoritmos de exclusão mútua distribuída (como Ricart-Agrawala ou baseados em token) introduzem latência e complexidade significativas, e devem ser evitados quando a semântica das operações permite.

15 Tolerância a Falhas de Tracker

15.1 Não há tracker líder, não há necessidade de eleição

No PeerSpot, todos os trackers são equivalentes e mantêm cópias sincronizadas do índice global. Não existe tracker primário nem hierarquia entre eles, portanto não há necessidade de algoritmo de eleição. Tanenbaum (Cap. 6.4) aponta que algoritmos de eleição são necessários apenas quando o sistema depende de um coordenador único, o que não é o caso aqui.

15.2 Fallback no peer

Cada peer mantém localmente uma lista ordenada de trackers conhecidos: o tracker principal e um ou mais trackers de backup. Quando o peer detecta que o tracker principal não responde (timeout na conexão duradoura), tenta o próximo da lista automaticamente, sem qualquer coordenação com outros nós da rede:

```
1 TRACKERS = [  
2     {"id": "tracker-1", "ip": "192.168.0.10", "porta": 8000}, #  
   principal  
3     {"id": "tracker-2", "ip": "192.168.0.11", "porta": 8000}, # backup  
4 ]  
5  
6 def conectar_tracker():  
7     for t in TRACKERS:  
8         try:  
9             return conectar(t["ip"], t["porta"])  
10        except TimeoutError:  
11            continue  
12    raise Exception("Nenhum tracker disponivel")
```

Listing 7: Lista de fallback no peer

A reorganização é imediata e local, o peer simplesmente passa a operar com o próximo tracker da lista, que já possui o índice completo sincronizado.

15.3 Reintegração do tracker e balanceamento de carga

Quando um tracker que estava fora do ar volta a funcionar, ele precisa: (1) recuperar o estado atual do índice e (2) receber de volta parte dos peers para distribuir a carga entre os trackers ativos.

Ao iniciar, o tracker entra no grupo multicast e envia uma mensagem `TRACKER_REJOIN`. Os trackers que recebem essa mensagem respondem com o estado completo de suas tabelas (`FULL_SYNC`), reconstruindo o índice do tracker reintegrado. Em seguida, cada tracker ativo cede uma fração de seus peers ao tracker reintegrado, enviando a cada peer cedido uma mensagem `REASSIGN_TRACKER` com o endereço do novo tracker:

```
1 # Tracker reintegrado anuncia retorno
2 TRACKER_REJOIN = {
3     "type": "TRACKER_REJOIN",
4     "tracker_id": "string",
5     "ip": "string",
6     "porta": "int"
7 }
8
9 # Tracker ativo cede peer ao tracker reintegrado
10 REASSIGN_TRACKER = {
11     "type": "REASSIGN_TRACKER",
12     "peer_nome": "string",
13     "novo_tracker_ip": "string",
14     "novo_tracker_porta": "int"
15 }
```

Listing 8: Mensagens de reintegração e balanceamento

O critério de balanceamento é simples: cada tracker ativo divide seu número atual de peers pelo número total de trackers na rede (incluindo o reintegrado) e cede o excedente. Isso distribui a carga de forma aproximadamente uniforme sem necessidade de coordenação centralizada.